第 15 讲：动态规划入门——保存重复子问题的答案

0. 本讲的核心问题

有些问题可以分解成更小的相同类型问题：

- 到第 `i` 级台阶有多少种走法？
- 偷到第 `i` 间房屋时最多得到多少钱？
- 凑出金额 `a` 最少需要多少枚硬币？
- 到网格位置 `(row, col)` 有多少条路径？

直接递归可能反复计算同一个子问题。

动态规划 DP 的核心是：

> 明确定义子问题的状态，保存其答案，并按照依赖顺序组合出更大问题的答案。

动态规划不是“看到最值就套数组”，而是一套推导过程：

→ → → →

---

1. 从斐波那契数列看重复计算

递归定义：

```
[python]
def fibonacci(n):
    if n <= 1:
        return n

    return fibonacci(n - 1) + fibonacci(n - 2)
```

计算 `fibonacci(5)` 时：

```
fib(5)
├── fib(4)
│   ├── fib(3)
│   └── fib(2)
└── fib(3)
    ├── fib(2)
    └── fib(1)
```

`fib(3)`、`fib(2)` 被反复计算，形成指数级调用。

两个特征提示动态规划：

1. 重叠子问题：相同状态被多次求解；
2. 最优子结构或可组合结构：大问题答案能由小问题答案得到。

---

2. 两种实现方式

2.1 自顶向下：记忆化搜索

仍按递归思路求解，但把计算结果保存起来：

```
[python]
def fibonacci(n):
    memo = {}

    def dp(x):
        if x <= 1:
            return x

        if x in memo:
            return memo[x]

        memo[x] = dp(x - 1) + dp(x - 2)
        return memo[x]

    return dp(n)
```

也可以使用：

```
[python]
from functools import cache

@cache
def fibonacci(n):
    if n <= 1:
        return n

    return fibonacci(n - 1) + fibonacci(n - 2)
```

记忆化搜索适合先写出自然递归关系。

2.2 自底向上：表格递推

先算小状态，再算大状态：

```
[python]
def fibonacci(n):
    if n <= 1:
        return n

    dp = [0] * (n + 1)
    dp[0] = 0
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

自底向上避免递归深度问题，计算顺序更明确。

---

3. DP 五步法

遇到动态规划题，先写以下五项，再编码。

第一步：定义状态

用一句完整的话定义：

```
dp[i]
```

状态定义必须包含范围和约束。例如：

```
dp[i]       i
```

不要只写“dp[i] 是答案”。

第二步：写状态转移

问最后一步从哪里来：

```
i
```

第三步：确定初始状态

转移无法继续向前推时，需要哪些已知值？

第四步：确定计算顺序

计算 `dp[i]` 时，它依赖的状态必须已经完成。

第五步：确定返回值

答案一定是 `dp[-1]` 吗？有时是最大值、最小值、整行或多个状态的组合。

---

4. 爬楼梯

每次可以走 1 或 2 级，问到第 `n` 级有多少种方法。

4.1 状态定义

```
dp[i]     i
```

4.2 最后一步分析

到达第 `i` 级的最后一步只有两种可能：

- 从 `i-1` 走 1 级；
- 从 `i-2` 走 2 级。

两类方案互不重叠：

```
dp[i] = dp[i-1] + dp[i-2]
```

4.3 初始状态

```
dp[1] = 1
dp[2] = 2
```

4.4 代码

```
[python]
def climb_stairs(n):
    if n <= 2:
        return n

    dp = [0] * (n + 1)
    dp[1] = 1
    dp[2] = 2

    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

时间 `O(n)`，空间 `O(n)`。

---

5. 空间优化：只保存真正需要的状态

`dp[i]` 只依赖前两个状态，不需要保存整张表：

```
[python]
def climb_stairs(n):
    if n <= 2:
        return n

    two_steps_before = 1
    one_step_before = 2

    for i in range(3, n + 1):
        current = one_step_before + two_steps_before
        two_steps_before = one_step_before
        one_step_before = current

    return one_step_before
```

空间降为 `O(1)`。

学习初期不要过早压缩。先用数组写清状态，确认正确后再观察依赖范围。

---

6. 打家劫舍：状态必须包含约束

相邻房屋不能同时偷，求最大金额。

6.1 状态定义

一种简洁定义：

```
dp[i]      i
```

注意这里的 `i` 是房屋数量，数组中第 `i` 间房屋下标是 `i-1`。

6.2 最后一个决定

考虑第 `i` 间房屋：

- 不偷：答案是 `dp[i-1]`；
- 偷：上一间不能偷，答案是 `dp[i-2] + nums[i-1]`。

取最大：

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
```

6.3 初始状态

```
dp[0] = 0
dp[1] = nums[0]
```

6.4 代码

```
[python]
def rob(nums):
    n = len(nums)

    if n == 0:
        return 0

    dp = [0] * (n + 1)
    dp[0] = 0
    dp[1] = nums[0]

    for i in range(2, n + 1):
        skip = dp[i - 1]
        take = dp[i - 2] + nums[i - 1]
        dp[i] = max(skip, take)

    return dp[n]
```

6.5 不变量

计算完 `dp[i]` 后：

> `dp[i]` 是前 `i` 间房屋所有合法选择中的最大金额。

状态转移完整枚举了最后一间“偷”或“不偷”两种互斥情况。

---

7. 最大子数组和

求连续子数组的最大和。

7.1 状态定义必须包含“以 i 结尾”

```
dp[i]     nums[i]
```

为什么不是“前 i 个数中的最大和”？

为了向右延伸连续区间，需要知道当前最优区间是否确实接在 `i` 位置。

7.2 转移

以 `i` 结尾的子数组：

- 从 `nums[i]` 重新开始；
- 接在以 `i-1` 结尾的最优子数组后。

```
dp[i] = max(nums[i], dp[i-1] + nums[i])
```

7.3 答案不是 `dp[-1]`

最大子数组可能在数组中间结束，因此答案是：

```
max(dp)
```

空间优化写法：

```
[python]
def max_subarray(nums):
    current = nums[0]
    best = nums[0]
```

```
    for i in range(1, len(nums)):
        current = max(nums[i], current + nums[i])
        best = max(best, current)

    return best
```

这里 `current` 就是压缩后的 `dp[i]`。

---

8. 不同路径：二维 DP

机器人从左上角到右下角，只能向右或向下。

8.1 状态定义

```
dp[row][col]      (row, col)
```

8.2 转移

到达当前格子的最后一步来自：

- 上方 `(row-1, col)`；
- 左方 `(row, col-1)`。

```
dp[row][col] = dp[row-1][col] + dp[row][col-1]
```

8.3 初始状态

第一行只能一直向右，第一列只能一直向下，因此全部为 1。

```
[python]
def unique_paths(rows, cols):
    dp = [[1] * cols for _ in range(rows)]

    for row in range(1, rows):
        for col in range(1, cols):
            dp[row][col] = dp[row - 1][col] + dp[row][col - 1]

    return dp[rows - 1][cols - 1]
```

复杂度：

```
O(rows × cols)
O(rows × cols)
```

8.4 一维压缩

当前行只依赖上一行和当前行左侧：

```
[python]
def unique_paths(rows, cols):
    dp = [1] * cols

    for row in range(1, rows):
        for col in range(1, cols):
            dp[col] = dp[col] + dp[col - 1]

    return dp[-1]
```

更新前 `dp[col]` 是上方路径数；`dp[col-1]` 已更新，是左方路径数。

---

9. 最小路径和

每个格子有代价，只能向右或向下，求最小路径和。

状态：

```
dp[row][col]
```

转移：

```
dp[row][col] = grid[row][col] + min(
    dp[row-1][col],
    dp[row][col-1]
)
```

```
[python]
def min_path_sum(grid):
    rows = len(grid)
    cols = len(grid[0])
    dp = [[0] * cols for _ in range(rows)]

    dp[0][0] = grid[0][0]

    for col in range(1, cols):
        dp[0][col] = dp[0][col - 1] + grid[0][col]

    for row in range(1, rows):
        dp[row][0] = dp[row - 1][0] + grid[row][0]

    for row in range(1, rows):
        for col in range(1, cols):
            dp[row][col] = grid[row][col] + min(
                dp[row - 1][col],
                dp[row][col - 1],
            )

    return dp[-1][-1]
```

二维 DP 最容易错在第一行和第一列的边界初始化。

也可以给表格外侧设置哨兵值，但学习初期分开初始化更清楚。

---

10. 零钱兑换：完全背包入门

给定硬币面值，每种可无限使用，求凑成 amount 的最少硬币数。

10.1 状态定义

```
dp[value]      value
```

10.2 转移

如果最后使用一枚面值为 `coin` 的硬币，那么之前要凑出：

```
value - coin
```

所以：

```
dp[value] = min(dp[value], dp[value-coin] + 1)
```

10.3 初始值与“不可能”状态

```
dp[0] = 0
```

其他金额一开始未知，可以设置为一个不可能优于答案的大值：

```
[python]
amount + 1
```

因为最多使用 `amount` 枚面值 1 的硬币；若连这都做不到，大值仍保留。

10.4 代码

```
[python]
def coin_change(coins, amount):
    impossible = amount + 1
    dp = [impossible] * (amount + 1)
    dp[0] = 0

    for value in range(1, amount + 1):
        for coin in coins:
            if coin <= value:
                dp[value] = min(
                    dp[value],
                    dp[value - coin] + 1,
                )

    if dp[amount] == impossible:
        return -1

    return dp[amount]
```

复杂度：

```
O(amount × len(coins))
O(amount)
```

---

11. 为什么贪心选择最大硬币不一定正确

硬币：

```
[1, 3, 4]
```

金额 `6`：

- 贪心：先选 4，再选 1、1，共 3 枚；
- 最优：3 + 3，共 2 枚。

局部最大的选择不保证全局最优。DP 会比较所有可能的最后一枚硬币，同时复用更小金额的最优答案。

使用贪心前需要证明贪心选择性质，不能只因为它“看起来最快”。

---

12. 最长递增子序列

子序列不要求连续。定义：

```
dp[i]   nums[i]
```

要接到 `nums[i]` 前面的元素必须满足：

```
j < i   nums[j] < nums[i]
```

```
[python]
def length_of_lis(nums):
    if not nums:
        return 0

    dp = [1] * len(nums)

    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)
```

为什么初始值是 1?

每个元素单独都能组成长度为 1 的递增子序列。

为什么答案是 `max(dp)` 而不是 `dp[-1]`?

最长递增子序列不一定以最后一个元素结束。

这个基础解法时间 `O(n²)`。还有 `O(n log n)` 解法，但入门阶段应先掌握状态定义。

---

13. 0/1 背包的核心结构

有若干物品，每个物品只能选一次；容量有限，求最大价值。

二维定义：

```
dp[i][capacity]      i         capacity
```

对第 `i` 个物品：

- 不选它；
- 容量足够时选它。

```
dp[i][c] = max(
    dp[i-1][c],
    dp[i-1][c-weight] + value
)
```

13.1 一维压缩为什么要倒序

```
[python]
def knapsack_01(weights, values, capacity):
    dp = [0] * (capacity + 1)

    for weight, value in zip(weights, values):
        for current_capacity in range(capacity, weight - 1, -1):
            dp[current_capacity] = max(
                dp[current_capacity],
                dp[current_capacity - weight] + value,
            )

    return dp[capacity]
```

容量倒序保证右侧的：

```
[python]
dp[current_capacity - weight]
```

仍来自处理当前物品之前的上一轮，因此当前物品不会被重复使用。

如果正序更新，同一物品可能在一轮中被使用多次，那会变成完全背包的语义。

这一点先理解，不要求现在记住所有背包变体。

---

14. 记忆化搜索与表格 DP 如何选择

更适合记忆化搜索

- 递归关系容易自然描述；
- 状态空间中只有一部分真正会被访问；
- 计算顺序难以直接确定；

- 需要从回溯逐步优化。

更适合表格 DP

- 状态范围规则；
- 依赖顺序明确；
- 希望避免递归深度；
- 需要进行空间压缩。

二者本质相同：都保证每个状态只计算有限次，并保存结果。

---

15. DP 与回溯、贪心的区别

回溯

枚举选择路径，常用于列出全部方案。状态通常在返回时撤销。

动态规划

把不同路径汇聚到同一状态，只保存这个状态所需的答案，不再关心完整选择历史。

贪心

每步只做当前看起来最优的选择，不回头，也不比较所有子问题。需要单独证明局部选择能导向全局最优。

例如打家劫舍中，不能简单总偷金额更大的相邻房屋；DP 比较“偷当前”和“不偷当前”两类完整子问题。

---

16. 怎样判断状态是否定义充分

一个好状态必须满足：

> 只知道状态中保存的信息，就足以计算未来，不需要重新查看被丢弃的历史细节。

例如股票问题中，如果只定义：

```
dp[i] =   i
```

可能不够，因为未来卖出时还需要知道当前是否持有股票。应增加状态维度：

```
dp[i][0]   i
dp[i][1]   i
```

状态不够时，不要硬写转移；先补充缺失信息。

---

17. 正确性怎样解释

动态规划的正确性通常来自两部分：

17.1 状态覆盖所有合法情况

例如打家劫舍的最后一间只有“偷”和“不偷”两种，互斥且完整。

17.2 转移使用的子问题已经最优

如果 `dp[i-2]` 不是前 `i-2` 间房的最优答案，那么用它构造的结果也无法保证最优。

这通常可以用数学归纳法理解：

1. 初始状态正确；
2. 假设更小状态正确；
3. 转移完整比较所有最后一步，因此当前状态正确。

面试时不必写正式证明，但应该能说出“最后一步有哪些互斥情况”。

---

18. 常见错误

18.1 状态定义模糊

必须写清 `dp[i]` 是否包含第 `i` 个元素、是否必须以 `i` 结尾、是否只考虑前 `i` 个元素。

18.2 下标与数量混淆

`dp[i]` 表示前 `i` 个元素时，当前数组元素通常是 `nums[i-1]`。

18.3 初始状态不符合定义

不要先抄 `dp[0] = 0`。从状态定义推导空集合、第一项应是多少。

18.4 计算顺序错误

使用某状态前，它必须已经计算完成。二维表可能需要从左上到右下，也可能相反。

18.5 返回错位置

最大子数组和、最长递增子序列返回 `max(dp)`，不一定是 `dp[-1]`。

18.6 最小值问题把未知状态初始化为 0

0 可能被错误当成更优解。最小值问题常用 `float("inf")` 或可靠的大值。

18.7 空间压缩后覆盖依赖

先画出当前状态依赖上一行还是当前行，再决定正序或倒序。

18.8 看见递归就加缓存

缓存键必须包含决定子问题答案的全部状态。如果遗漏变量，不同子问题会错误共享结果。

---

19. 本讲检查表

```
1.
2.
3. dp[i]   dp[i][j]
4.
5.
6.
7.
8.
9.
10.
11.      O(n)       O(1)
12.
```

20. 本讲总结

1. 动态规划通过保存子问题答案避免重复计算。
2. 标准推导顺序是状态、转移、初始化、计算顺序和返回值。
3. 记忆化搜索与自底向上 DP 本质相同，只是求解顺序不同。
4. 状态定义必须包含未来决策所需的全部信息。
5. “最后一步”分析常能自然推出状态转移。
6. 空间优化应在正确的一维或二维 DP 之后进行。
7. DP 的复杂度通常是“状态数量 × 每个状态的转移成本”。

---

21. 学完 15 讲后的训练顺序

完成本讲后，不要立刻追求刷完 Hot 100。建议按模式各选代表题达到能够独立完成：

```
/

/BFS

        BST
  k     k
```

每个模式先让一两道代表题达到 C 级，再扩展到外观不同的题。真正目标是看到新题时能够说出：